

LESSON 4: CONDITIONAL STATEMENTS & CONTROL FLOW

Python Programming Foundation Series — Comprehensive In-Depth Study Notes

1. CONTROL FLOW & DECISION-MAKING ARCHITECTURE

In standard procedural programming, an interpreter processes instructions linearly from top to bottom (sequential execution). However, real-world algorithms require dynamic behavior—executing certain routines only when specific criteria are met, selecting between alternatives, or skipping invalid operations entirely.

Control Flow Statements break this rigid linear progression. By evaluating mathematical and logical predicates, they establish branching execution trees:

Input State $\rightarrow$ Predicate Evaluation [$P \in \{\text{True}, \text{False}\}$] $\rightarrow$ Execution Branch A or B

Python provides four foundational branching constructs:

1. The Single-Branch if Statement: Evaluates a single predicate and executes a block only when True.
2. The Dual-Branch if-else Construct: Divides execution into two mutually exclusive pathways.
3. The Multi-Branch if-elif-else Ladder: Evaluates a priority-ordered sequence of independent conditions.
4. Conditional Expressions (Ternary Operator): Inline expressions designed for assignments based on conditions.

2. INDENTATION, SCOPE & CODE BLOCKS

Unlike C, C++, Java, and JavaScript—which designate blocks of code using explicit curly braces { ... } and terminate statements with semicolons ;—Python implements the Off-Side Rule.

C-Style Block Demarcation:

```
if (score >= 90) {  
    grade = "A";  
    printf("Passed");  
}
```

Python Block Demarcation:

if score >= 90:

```
    grade = "A"    # 4 spaces of indentation
```

```
    print("Passed") # 4 spaces of indentation
```

The Rules of Indentation

- **The Colon (:):** Every conditional header (if, elif, else) must terminate with a colon. This indicates to the lexer/parser that an indented block follows.
- **Standard Width:** PEP 8 strictly mandates 4 spaces per indentation level.
- **No Tabs/Spaces Mixing:** Mixing tabs and spaces raises a compile-time `TabError`.
- **Block Termination:** A block automatically terminates when the indentation level returns to that of an enclosing block.

3. RELATIONAL & LOGICAL OPERATORS

Conditional statements evaluate boolean predicates built using relational and logical operators.

3.1 Relational (Comparison) Operators

Operator	Meaning	Example Expression	Result
<code>==</code>	Value Equality (checks data equivalent)	<code>10 == 10</code>	True
<code>!=</code>	Value Inequality	<code>10 != 5</code>	True
<code>></code>	Strictly Greater Than	<code>15 > 20</code>	False
<code><</code>	Strictly Less Than	<code>5 < 12</code>	True
<code>>=</code>	Greater Than or Equal To	<code>18 >= 18</code>	True
<code><=</code>	Less Than or Equal To	<code>7 <= 6</code>	False

3.2 Logical Operators (Boolean Connectives)

- **and (Logical Conjunction):** Returns True if and only if both operands evaluate to true.

- **or (Logical Disjunction):** Returns True if at least one operand evaluates to true.
- **not (Logical Inversion):** Unary operator that inverts the truth value (not True yields False).

3.3 Short-Circuit Evaluation

Python's logical operators use short-circuit evaluation:

- **For A and B:** If A evaluates to falsy, Python returns A immediately without ever evaluating B (because the entire conjunction is already guaranteed to be false).
- **For A or B:** If A evaluates to truthy, Python returns A immediately without ever evaluating B (because the entire disjunction is already guaranteed to be true).

Python

Short-circuiting protects against ZeroDivisionError:

```
denominator = 0
```

If denominator == 0 is False, the second expression is never checked:

```
if denominator != 0 and (100 / denominator) > 2:
```

```
    print("Safe division performed")
```

```
else:
```

```
    print("Zero denominator safely intercepted without crashing!")
```

3.4 Chained Comparisons

Python allows mathematical comparison chaining natively:

Python

Instead of writing:

```
if x >= 10 and x <= 50:
```

```
    pass
```

Python allows natural mathematical chaining:

```
if 10 <= x <= 50:
```

```
    pass
```

4. BRANCHING TAXONOMY: DEEP DIVE

4.1 The Single-Branch if

Executes code conditionally. If the predicate evaluates to False, the indented suite is bypassed entirely.

Python

```
age = 20
```

```
if age >= 18:
```

```
    print("Eligible to cast vote.")
```

4.2 The Dual-Branch if-else

Guarantees that exactly one of two mutually exclusive blocks executes:

Python

```
number = 17
```

```
if number % 2 == 0:
```

```
    print(f"{number} is Even.")
```

```
else:
```

```
    print(f"{number} is Odd.")
```

4.3 The Multi-Branch if-elif-else Ladder

When testing multiple sequential conditions, using separate if statements is inefficient and buggy because every independent if runs its check. The elif (short for *else-if*) ladder creates a single linked decision tree:

- Python evaluates conditions sequentially from top to bottom.
- The first condition that evaluates to True executes its block.
- All subsequent elif and else blocks are skipped immediately.
- The trailing else acts as an optional catch-all fallback.

Python

```
score = 85
```

```
if score >= 90:
    grade = "A+"
elif score >= 80:
    grade = "A" # Executes; matches first!
elif score >= 70:
    grade = "B" # Skipped even though 85 >= 70 is True!
elif score >= 60:
    grade = "C" # Skipped
else:
    grade = "F" # Skipped
```

4.4 Nested Conditions

An if statement can reside within the body of an outer if or else block:

Python

```
account_balance = 5000
withdrawal_amount = 1200
is_account_active = True

if is_account_active:
    if withdrawal_amount <= account_balance:
        account_balance -= withdrawal_amount
        print(f"Withdrawal authorized. Balance: ${account_balance}")
    else:
        print("Transaction declined: Insufficient funds.")
else:
    print("Transaction declined: Account is suspended.")
```

5. CONDITIONAL EXPRESSIONS (TERNARY OPERATOR)

Python provides an inline expression syntax for evaluating conditions and returning values:

Syntax: `x = value_if_true if condition else value_if_false`

Key Differences from Statement Blocks:

- **Expression vs Statement:** Standard if-else statements do not return values; conditional expressions evaluate directly to a value and can be passed into functions or assigned directly to variables.
- **Mandatory else:** The else clause is syntactically mandatory in a ternary expression.

Python

```
age = 20
```

Standard if-else statement:

```
if age >= 18:
```

```
    ticket_category = "Adult"
```

```
else:
```

```
    ticket_category = "Minor"
```

Idiomatic conditional expression:

```
ticket_category = "Adult" if age >= 18 else "Minor"
```

Usable directly within function calls:

```
print(f"Access granted to: {'Adult' if age >= 18 else 'Minor'}")
```

6. THE TRUTHY AND FALSY EVALUATION MODEL

In Python, every object has an inherent truth value (truthiness) and can be evaluated directly within a conditional statement without explicit comparison to True or False.

6.1 Standard Falsy Values

Python defines the following specific objects and values as Falsy:

Category	Falsy Primitive / Collection
Constants	None, False
Numeric Zeros	0, 0.0, 0j, Decimal(0), Fraction(0, 1)
Empty Sequences	"" (empty string), () (empty tuple), [] (empty list), range(0)
Empty Mappings / Sets	{ } (empty dict), set(), frozenset()
Custom Classes	Objects whose <code>__bool__()</code> or <code>__len__()</code> returns False or 0

Everything else in Python evaluates to Truthy.

6.2 Idiomatic Pythonic Truth Checks

Python

❌ UN-PYTHONIC & REDUNDANT:

```
if len(users_list) > 0:
```

```
    pass
```

```
if user_input == True:
```

```
    pass
```

```
if active_token != "":
```

```
    pass
```

✅ IDIOMATIC (PYTHONIC):

```
if users_list:
```

```
    # Runs only if collection contains items
```

```
pass
```

```
if user_input:
```

```
    # Runs only if True
```

```
    pass
```

```
if active_token:
```

```
    # Runs only if string is not empty
```

```
    pass
```

7. COMPLETE PRODUCTION SCRIPT

Below is a single, self-contained Python script implementing relational expressions, multi-tier tax evaluation, nested logic, truthy checks, and practical edge-case handling:

Python

```
"""
```

Lesson 4: Conditional Statements & Flow Control Architecture

Demonstrating relational operators, logical expressions, truthiness, nested decision trees, and real-world domain validation.

```
"""
```

```
# =====
```

1. BASIC RELATIONAL LOGIC & CONDITIONAL BRANCHING

```
# =====
```

```
print("--- 1. Relational & Basic Branching ---")
```

```
voter_age = 20
```

```
if voter_age >= 18:
```



```
print(f"Status: Age {voter_age} satisfies legal threshold. Voting allowed.")
else:
    print(f"Status: Age {voter_age} is below threshold. Voting prohibited.")
```

Parity Determination

```
number = 15
```

```
if number % 2 == 0:
```

```
    print(f"The number {number} is Even.")
```

```
else:
```

```
    print(f"The number {number} is Odd.")
```

```
# =====
```

2. MULTI-BRANCH ELIF LADDER (ACADEMIC GRADING ENGINE)

```
# =====
```

```
print("\n--- 2. Academic Grading Engine ---")
```

```
test_score = 78.5
```

```
if test_score >= 90.0:
```

```
    grade_designation = "A+ (Outstanding)"
```

```
elif test_score >= 80.0:
```

```
    grade_designation = "A (Excellent)"
```

```
elif test_score >= 70.0:
```

```
    grade_designation = "B (Proficient)"
```

```
elif test_score >= 60.0:
```

```
    grade_designation = "C (Satisfactory)"
```

```
elif test_score >= 50.0:
```

```

    grade_designation = "D (Marginal Pass)"

else:

    grade_designation = "F (Failing)"


print(f"Score Record: {test_score}% -> Awarded: {grade_designation}")


# =====

# 3. NESTED CONDITIONS: FINANCIAL TRANSACTION AUTHORIZATION

# =====

print("\n--- 3. Financial Transaction Guard ---")

customer_authenticated = True

account_verified = True

available_balance = 12500.00

requested_withdrawal = 3000.00


if customer_authenticated:

    if account_verified:

        if requested_withdrawal <= available_balance:

            available_balance -= requested_withdrawal

            print(f"Approved: ${requested_withdrawal:.2f} dispensed.")

            print(f"Remaining Capital: ${available_balance:.2f}")

        else:

            print("Declined: Requested funds exceed available balance.")

    else:

        print("Declined: KYC verification incomplete.")

else:

```

```

    print("Declined: Authentication handshake failed.")

# =====

# 4. CONDITIONAL EXPRESSIONS (TERNARY OPERATOR)

# =====

print("\n--- 4. Inline Conditional Expressions ---")

account_tier = "Premium" if available_balance >= 10000.00 else "Standard"

print(f"Current Account Standing: {account_tier}")

# Multi-level inline ternary (used with care in production)

numeric_target = 0

sign_indicator = "Positive" if numeric_target > 0 else ("Zero" if numeric_target == 0 else
"Negative")

print(f"Value {numeric_target} is categorized as: {sign_indicator}")

# =====

# 5. TRUTHY & FALSY ITERATION & INFERENCE

# =====

print("\n--- 5. Truth Value Verification ---")

test_structures = [0, 0.0, "", "Database", None, [], [10, 20], {}, {"key": "val"}]

for element in test_structures:
    evaluation = "Truthy" if element else "Falsy"
    print(f"Value: {repr(element):<15} -> Result: {evaluation}")

# =====

```

6. PRACTICAL CASE STUDY: PROGRESSIVE UTILITY BILLING

```
# =====
```

```
print("\n--- 6. Case Study: Progressive Power Tariff ---")
```

```
def calculate_utility_invoice(units_consumed: float) -> float:
```

```
    """
```

```
    Computes utility invoice based on tiered progressive rates:
```

- 0 to 100 units: \$1.50 per unit
- 101 to 200 units: \$2.00 per unit
- 201 to 300 units: \$3.00 per unit
- Above 300 units: \$4.50 per unit

```
    """
```

```
    if units_consumed <= 100:
```

```
        total_bill = units_consumed * 1.50
```

```
    elif units_consumed <= 200:
```

```
        total_bill = (100 * 1.50) + ((units_consumed - 100) * 2.00)
```

```
    elif units_consumed <= 300:
```

```
        total_bill = (100 * 1.50) + (100 * 2.00) + ((units_consumed - 200) * 3.00)
```

```
    else:
```

```
        total_bill = (100 * 1.50) + (100 * 2.00) + (100 * 3.00) + ((units_consumed - 300) * 4.50)
```

```
    return total_bill
```

```
consumption_record = 350
```

```
invoice_amount = calculate_utility_invoice(consumption_record)
```

```
print(f"Consumption: {consumption_record} kWh")
```

```
print(f"Calculated Tiered Utility Charge: ${invoice_amount:.2f}")
```

```
# Leap Year Calculation
```

```
test_year = 2024
```

```
is_leap = (test_year % 4 == 0 and test_year % 100 != 0) or (test_year % 400 == 0)
```

```
print(f"\nYear {test_year} Leap Year Check: {'Yes' if is_leap else 'No'}")
```

8. CODE ARCHITECTURE BREAKDOWN: STEP-BY-STEP FLOW

The execution cycle inside the script runs through several core mechanisms:

Phase 1: Boolean Predicate Resolution

In `if voter_age >= 18:`, the relational operator `>=` compares the memory value of `voter_age` (20) with the integer literal 18. This resolves to a singleton boolean object `True`. The PVM evaluates the branch jump instruction and enters the indented block.

Phase 2: Sequential Traversal of `if-elif-else`

In the academic grading section, `test_score = 78.5`:

1. `test_score >= 90.0` → `78.5 >= 90.0` evaluates to `False`. The PVM skips to the first `elif`.
2. `test_score >= 80.0` → `78.5 >= 80.0` evaluates to `False`. The PVM skips to the second `elif`.
3. `test_score >= 70.0` → `78.5 >= 70.0` evaluates to `True`.
4. The assignment `grade_designation = "B (Proficient)"` executes.
5. The execution pointer immediately exits the entire structure, skipping checks for `>= 60.0`, `>= 50.0`, and the `else` fallback.

Phase 3: Short-Circuiting in the Leap Year Check

The leap year logic:

Python

```
is_leap = (test_year % 4 == 0 and test_year % 100 != 0) or (test_year % 400 == 0)
```

1. `test_year % 4 == 0` evaluates to `True` ($2024 \div 4$ has no remainder).
2. Because the first part of `and` is `True`, it checks `test_year % 100 != 0`, which is also `True` ($2024 \div 100$ leaves 24).

3. The left-hand side of the or is now True and True → True.
4. Short-circuiting takes effect: Because the left side of the or is True, Python skips evaluating `test_year % 400 == 0` entirely, instantly resolving `is_leap` to True.

9. COMMON DEVELOPER ANTI-PATTERNS & PITFALLS

Anti-Pattern 1: The Assignment-in-Condition Confusion

Python

❌ COMMON IN C/JAVA CONVERSIONS: `SyntaxError`

```
if score = 100:
```

```
    print("Perfect score!")
```

✅ CORRECT: Use the equality operator `==`

```
if score == 100:
```

```
    print("Perfect score!")
```

Why this fails: In Python, the single equals sign `=` is strictly an assignment statement. Statements cannot be evaluated as expressions within an if condition. (Note: Python 3.8+ supports the walrus operator `:=` for assignment expressions, e.g., `if (n := len(items)) > 0:`, but `simple = remains` a syntax error).

Anti-Pattern 2: The "Arrow Anti-Pattern" (Deep Nesting)

Nesting conditions 5 or 6 levels deep makes code unreadable and hard to test:

Python

❌ SMELL: Hard to read arrow-shaped indentation

```
if user.is_authenticated:
```

```
    if user.has_active_plan:
```

```
        if item.in_stock:
```

```
            if payment_method.is_valid:
```

```
                execute_order()
```

 **REFACTORED: Guard clauses / Early returns**

```
if not user.is_authenticated:
```

```
    return "User unauthenticated"
```

```
if not user.has_active_plan:
```

```
    return "No active subscription"
```

```
if not item.in_stock:
```

```
    return "Inventory unavailable"
```

```
if not payment_method.is_valid:
```

```
    return "Payment failed"
```

```
execute_order()
```

Anti-Pattern 3: Chaining Separate if Statements Mistakenly

Python

 **LOGICAL BUG: Multiple blocks execute unintentionally!**

```
x = 85
```

```
if x >= 80:
```

```
    print("Pass")
```

```
if x >= 70:
```

```
    print("Passed with Merit") # THIS ALSO PRINTS!
```

 **CORRECT: Use elif to enforce mutual exclusivity**

```
if x >= 80:
```

```
    print("Pass")
```

```
elif x >= 70:
```

```
    print("Passed with Merit")
```

10. TECHNICAL INTERVIEW & RECAP CHEAT SHEET

High-Yield Questions & Answers

Q1: What is the difference between == and is in Python conditional statements?

Answer: == checks for value equality (calling the object's internal `__eq__()` method to determine if the data is equivalent). In contrast, `is` checks for identity equality (verifying whether both variables point to the exact same physical memory address: `id(a) == id(b)`). Always use == for comparing numbers, strings, and collections, and reserve `is` for singleton comparisons like `if val is None:`.

Q2: What is Short-Circuit Evaluation and how does it optimize performance?

Answer: Short-circuiting stops evaluating a boolean expression as soon as the final result is determined. In an `or` statement, if the first condition is `True`, the rest are skipped. In an `and` statement, if the first condition is `False`, the rest are skipped. This saves CPU time when evaluating expensive functions and guards against runtime crashes (like null pointer dereferencing or zero division).

Q3: How does Python evaluate the truthiness of custom class instances?

Answer: When Python checks an object in a boolean context, it first calls the class's `__bool__()` magic method. If `__bool__()` is not defined, Python falls back to `__len__()` (where a length of 0 is `False`, and `>0` is `True`). If neither method is implemented, the object instance defaults to `True`.

Q4: Can a single conditional expression (ternary operator) contain multiple `elif` branches?

Answer: While Python has no literal `elif` keyword for inline expressions, you can chain ternary expressions: `value1 if cond1 else (value2 if cond2 else fallback)`. However, deep chaining is discouraged in production code because it degrades readability.